

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Вычислительная математика»
Тема: Метод бисекции

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Нахождение корня уравнения методом бисекции с заданной точностью Eps, исследование зависимости числа итераций от точности Eps при заданном изменении Eps, исследование чувствительности к ошибкам в исходных данных.

Задание.

Используя программы - функции BISECT и Round из файла methods.cpp, найти корень уравнения $f(x) = 0$ методом бисекции с заданной точностью Eps, исследовать зависимость числа итераций от точности Eps при изменении Eps от 0.1 до 0.000001, исследовать обусловленность метода (чувствительность к ошибкам в исходных данных).

Вариант 17: $f(x) = e^x - \arccos(\sqrt{x})$

Порядок выполнения работы:

- 1) Графически или аналитически отделить корень уравнения $f(x) = 0$ (т.е. найти отрезки [Left, Right], на которых функция $f(x)$ удовлетворяет условиям теоремы Коши).
- 2) Составить подпрограмму вычисления функции $f(x)$.
- 3) Составить головную программу, содержащую обращение к подпрограмме $f(x)$, BISECT, Round и индикацию результатов.
- 4) Провести вычисления по программе. Построить график зависимости числа итераций от Eps.
- 5) Исследовать чувствительность метода к ошибкам в исходных данных. Ошибки в исходных данных моделировать с использованием программы Round, округляющей значения функции с заданной точностью Delta.

Теоретические положения.

Если найден отрезок $[a, b]$, такой, что $f(a)f(b) < 0$, существует точка c , в которой значение функции равно нулю, т.е. $f(c) = 0$, $c \in (a, b)$. Метод бисекции состоит в построении последовательности вложенных друг в друга отрезков, на концах которых функция имеет разные знаки. Каждый последующий отрезок получается делением пополам предыдущего. Процесс построения последовательности отрезков позволяет найти нуль функции $f(x)$ (корень уравнения $f(x) = 0$ с любой заданной точностью.

Рассмотрим один шаг итерационного процесса. Пусть на $(n-1)$ -м шаге найден отрезок $[a_{n-1}, b_{n-1}] \subset [a, b]$, такой, что $f(a_{n-1})f(b_{n-1}) < 0$. Разделим его пополам точкой $\xi = (a_{n-1} + b_{n-1})/2$ и вычислим $f(\xi)$. Если $f(\xi) = 0$, то $\xi = (a_{n-1} + b_{n-1})/2$ - корень уравнения. Если $f(\xi) \neq 0$, то из двух половин отрезка выбирается та, на концах которой функция имеет противоположные знаки, поскольку искомый корень лежит на этой половине, т.е.

$$a_n = a_{n-1}, b_n = \xi, \text{ если } f(\xi)f(a_{n-1}) < 0 ;$$

$$a_n = \xi, b_n = b_{n-1}, \text{ если } f(\xi)f(a_{n-1}) > 0 .$$

Если требуется найти корень с точностью ξ , то деление пополам продолжается до тех пор, пока длина отрезка не станет меньше 2ξ . Тогда координата середины отрезка есть значение корня с требуемой точностью ξ .

Метод бисекции является простым и надежным методом поиска простого корня уравнения $f(x) = 0$ (простым называется корень $x = c$ дифференцируемой функции $f(x)$, если $f(c) = 0$ и $f'(c) \neq 0$). Этот метод сходится для любых непрерывных функций $f(x)$, в том числе недифференцируемых. Скорость его сходимости невысока. Для достижения точности ξ необходимо совершить $N \approx \log_2(b-a)/\xi$ итераций. Это означает, что для получения каждых трех верных десятичных знаков необходимо совершить около 10 итераций.

Выполнение работы.

1) Корень уравнения $f(x) = 0$ для $f(x) = e^x - \arccos(\sqrt{x})$ был найден графически при помощи графического калькулятора Desmos. Полученный график представлен на рисунке 1.

Рассмотрим отрезок $[0.1, 0.2]$. Функция монотонно возрастающая. Из графика видно, что $f(0.1) < 0$, $f(0.2) > 0$. Следовательно на данном отрезке функция удовлетворяет условиям Коши.

Производная функции $f'(x) = e^x + 1/(2\sqrt{x}\sqrt{1-x})$ на заданном промежутке > 0 .

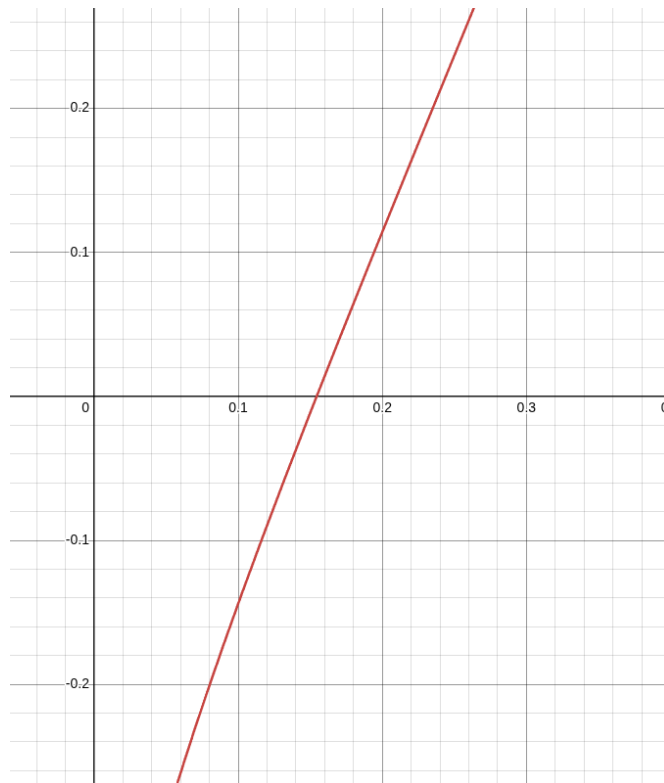


Рисунок 1 — График функции $f(x) = e^x - \arccos(\sqrt{x})$

2) `double F(double x)` — функция, принимающая значение x и возвращающая значение функции $f(x) = e^x - \arccos(\sqrt{x})$

3) В функции `main` задаются начальные значения границ отрезка: $[0.1, 0.2]$. Данные значения удовлетворяют условиям Коши, а также получившийся подотрезок больше $2 \cdot \text{Eps}$.

4) Затем в цикле выполняется функция бисекции для значений точности `Eps` от 0.1 до 0.000001. Полученные результаты выводятся. Данный вывод получает на вход файл `plots.py`. В данном файле при помощи python-модуля

matplotlib строится график зависимости числа итераций от точности Eps. Для наглядности шкала по оси абсцисс — логарифмическая. Из графика хорошо видно, что рост количества итераций при увеличении точности Eps — логарифмический.

Сами измерения представлены в таблице 1.

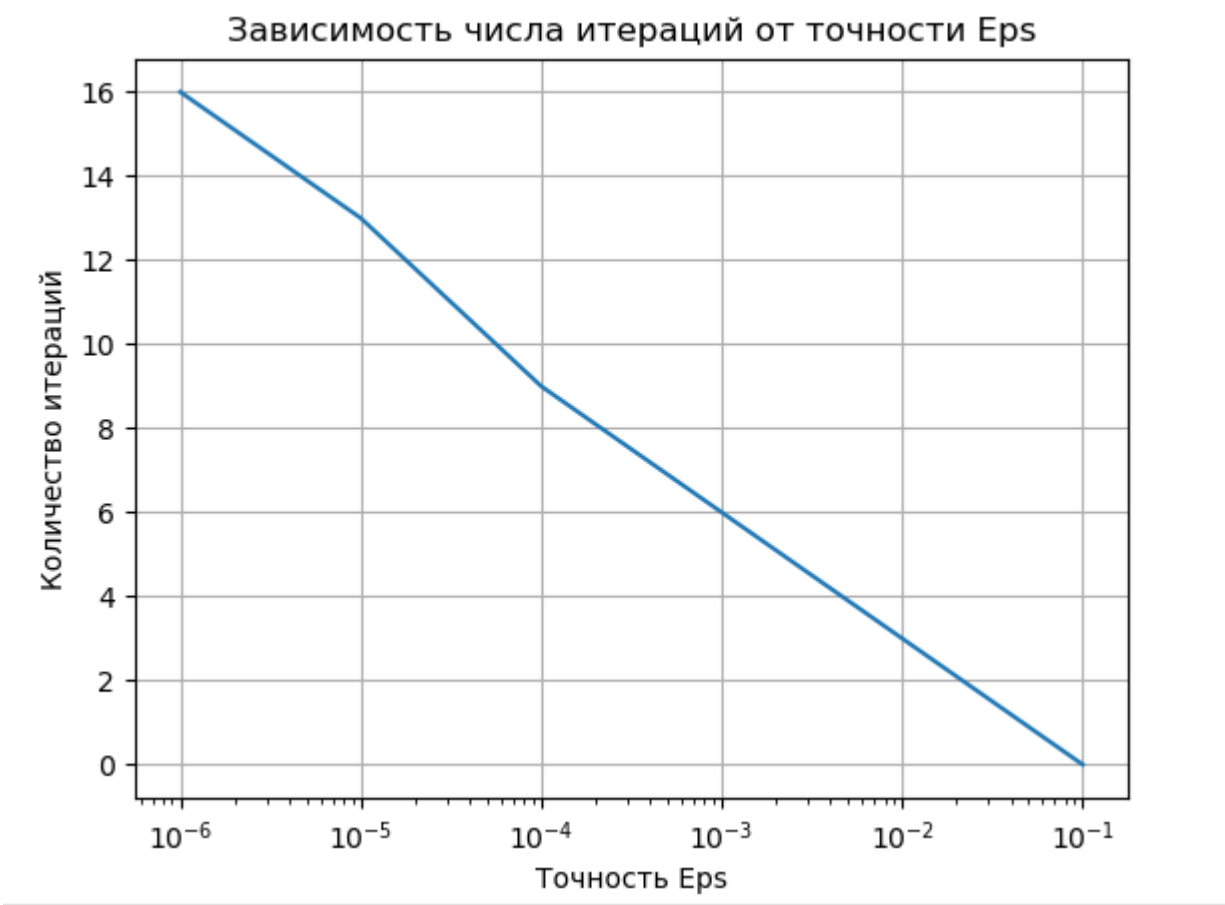


Рисунок 2 — График зависимости числа итреаций от точности Eps

Таблица 1 — Исследование зависимости числа итераций от точности Eps

Точность Eps	Число итераций	Приблженный корень
0.1	0	0.15
0.01	3	0.15625
0.001	6	0.15390625
0.0001	9	0.15439453125
0.00001	13	0.154400634765625

0.000001	16	0.154405975341797

5) Далее было произведено исследование чувствительности метода к ошибкам в исходных данных. Для этого была заведена глобальная переменная CurrentDelta, отвечающая за значение погрешности. Если она больше нуля, то в функции F вычисленное значение функции $f(x) = e^x - \arccos(\sqrt{x})$ округляется при помощи функции Round с заданной точностью.

В функции main() в цикле выполняется функция bisection для погрешностей от 10^{-3} до 10^{-8} . Результаты измерений числа итераций и вычисленного корня подаются на вход файлу plots.py, в котором строится график зависимости вычисленного приближенного корня от погрешности Delta (см. рис. 3). Шкалы — линейные. Из графика четко видно, что рост точности вычисления корня зависит линейно от величины погрешности Delta и при минимальной погрешности совпадает с вычисленным ранее корнем. Полученные измерения представлены в таблице 2.

Рисунок 3 — Зависимость вычисленного корня от погрешности Delta (см. ниже)

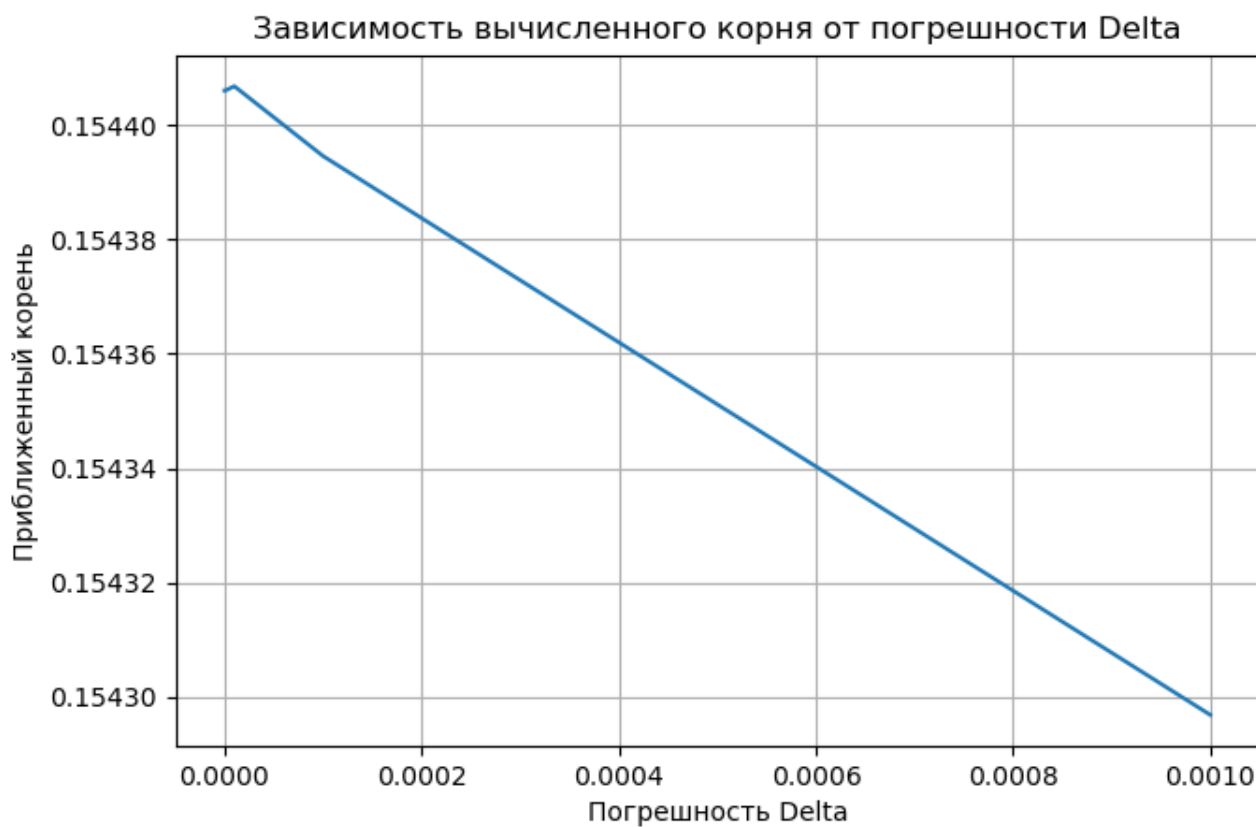


Таблица 2 — Исследование чувствительности к ошибкам исходных данных

Погрешность Delta	Число итераций	Приблизженный корень
0.001	7	0.154296875
0.0001	9	0.15439453125
0.00001	12	0.15440673828125
0.000001	16	0.154405975341797
0.0000001	16	0.154405975341797
0.00000001	16	0.154405975341797

Разработанный программный код см. в приложении А.

Выводы.

В ходе лабораторной работы был найден корень уравнения методом бисекции с заданной точностью Eps . Вычисленный результат совпал с результатом, вычисленным при помощи графического калькулятора. Было произведено исследование зависимости числа итераций от точности Eps при заданном изменении Eps , а также исследование чувствительности к ошибкам в исходных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: lb1.cpp

```
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
```

```
double CurrentDelta = 0.0;
```

```
double F(double x);
double Round(double X, double Delta);
double BISECT(double Left, double Right, double Eps, int &N);
```

```
double F(double x)
{
    double raw = exp(x) - acos(sqrt(x));
    if (CurrentDelta > 0.0)
        return Round(raw, CurrentDelta);
    else
        return raw;
}
```

```
double Round(double X, double Delta)
{
    if (Delta <= 1E-9)
    {
        puts("Неверное задание точности округления\n");
        exit(1);
    }
    if (X > 0.0)
        return Delta * (long)((X / Delta) + 0.5);
    else
        return Delta * (long)((X / Delta) - 0.5);
}
```

```
double BISECT(double Left, double Right, double Eps, int &N)
{
    double E = fabs(Eps) * 2.0;
    double FLeft = F(Left);
    double FRight = F(Right);
    double X;
    double Y;
```

```
    if (FLeft * FRight > 0.0)
    {
        printf("Ошибка: знаки функции на концах [%lf, %lf]
одинаковы!\n", Left, Right);
        exit(1);
    }
```

```
    N = 0;
    while ((Right - Left) >= E)
    {
        X = (Right + Left) / 2.0;
```

```

        Y = F(X);

        if (Y == 0.0) return X;

        if (Y * FLeft < 0.0)
            Right = X;
        else
        {
            Left = X;
            FLeft = Y;
        }
        N++;
    }
    return (Left + Right) / 2.0;
}

int main()
{
    double Left = 0.1;
    double Right = 0.2;

    double EpsValues[] = {0.1, 0.01, 0.001, 0.0001, 0.00001,
0.000001};
    int numEps = sizeof(EpsValues) / sizeof(EpsValues[0]);

    printf("%d\n", numEps);

    CurrentDelta = 0.0;

    for (int i = 0; i < numEps; i++)
    {
        double eps = EpsValues[i];
        int iter;
        double x = BISECT(Left, Right, eps, iter);
        printf("%.15lf %d %.15lf\n", eps, iter, x);
    }

    double EpsFixed = 1e-6;
    double DeltaValues[] = {1e-3, 1e-4, 1e-5, 1e-6, 1e-7, 1e-8};
    int numDelta = sizeof(DeltaValues) / sizeof(DeltaValues[0]);

    printf("%d\n", numDelta);

    for (int i = 0; i < numDelta; i++)
    {
        CurrentDelta = DeltaValues[i];
        int iter;
        double root = BISECT(Left, Right, EpsFixed, iter);
        printf("%.15lf %d %.15lf\n", CurrentDelta, iter, root);
    }

    return 0;
}

```

Название файла: plots.py

from matplotlib import pyplot as plt

```

def plot(x_data, y_data, x_label, y_label, name, file_name,
is_log_xscale = False):
    plt.plot(x_data, y_data)
    plt.grid()
    if is_log_xscale:
        plt.xscale('log')
    plt.xlabel(x_label)
    plt.ylabel(y_label)
    plt.title(name)
    plt.savefig(file_name)
    plt.show()

def main():
    eps_values = []
    n_values = []
    delta_values = []
    x_values = []

    print("Исследование зависимости числа операций от точности
eps");

    numEps = int(input())
    for _ in range(numEps):
        eps, n, x = [float(el) for el in input().split()]
        print(f"eps = {eps}, n = {n}, x = {x}")
        eps_values.append(eps)
        n_values.append(n)

        plot(eps_values, n_values, "Точность Eps", "Количество
итераций", "Зависимость числа итераций от точности Eps", "eps_n.png",
is_log_xscale=True)

    print("\nИсследование чувствительности к ошибкам исходных
данных")

    numDelta = int(input())
    for _ in range(numDelta):
        delta, n, x = [float(el) for el in input().split()]
        print(f"delta = {delta}, n = {n}, x = {x}")
        delta_values.append(delta)
        x_values.append(x)

        plot(delta_values, x_values, "Погрешность Delta", "Приближенный
корень", "Зависимость вычисленного корня от погрешности Delta",
"delta_x.png")

if __name__ == "__main__":
    main()

```